



Text encoding is simply a piece of cake

OS that's running it. This setting can be changed by setting the file encoding option to UTF-8. This type of encoding is largely platform independent and people can work on the same file using different OSes without text file encoding fiddling with special characters etc. Change the setting by going to Preferences > General > Workspace. Click on the other radio button and choose 'UTF-8'.



Find variables

Finding an instance of a variable is extremely useful while coding. There's a neat way of doing the above in the Eclipse IDE. Simply right-click the variable and go to references > Workspace. Doing this will display instances of the variable even from other Java classes. There's also a way to find where the particular variable is declared first. Simply right-click the variable and press [F3].



Format only edited lines

Though Eclipse has a great code formatter, it might be problematic when a user is dealing with multiple versions of the same file. This problem can be dealt with by allowing Eclipse to only format edited lines on the 'save' option. Enabling this option allows the user to format code and also retain the option of seeing what

exactly was changed in the code later on. To do the above, simply go to Preferences > Java > Editor > Save Actions and click on the 'format edited lines' radio button.



Newer version

In case your version of Eclipse has become outdated and you want to update to a newer build

of Eclipse IDE you can easily do so without corrupting the current installation. To do this, download the archive of the Eclipse version you want to update to. In your current installation go to File > Import > From Existing Installation and add the path of the downloaded archive. Doing this will update the Eclipse build as desired and manage installed plug-ins as well. On restarting Eclipse, the changes will have been seamlessly integrated.



Speed up performance

Performance is an important parameter for an IDE and lack of speed can be a dampener for code development. There are

quite a few ways of speeding up Eclipse performance. Below are a couple of quick and easy fixes for the same.

It's always a good idea to know which plug-in does what and disable the useless ones. Go to Preferences and type 'startup' in the search field and review the list of applications under 'Startup and Shutdown'. Disabling unnecessary plug-ins can really aid the IDE's performance.

Another small thing you can do is limit the number of editors that the IDE uses during general usage. Each editor consumes memory and hogs computing power for tasks such as data validation. Go to Preferences > Editors and enter a small value in the 'number of editors' option box.

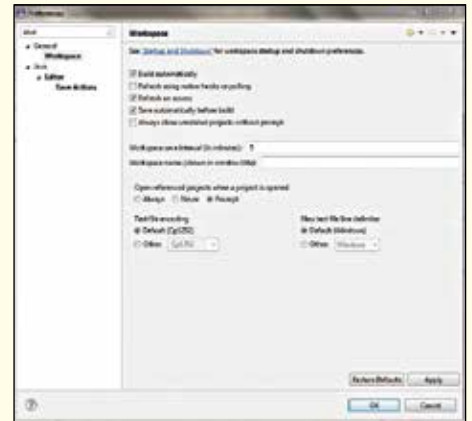
The above mentioned tips can be easily implemented and will improve the IDE's performance by a decent margin. Other options like getting a Solid State Drive and playing around with memory directives in the .eni file

also vastly improve IDE speed and responsiveness.



Automatically save before build

A very common action seen in most programmers, amateur or expert, is the compulsive need to hit the 'save' action button time and again. Enable the 'Save



Save before build. Always.

automatically before build' option by going to Preferences > General > Workspace

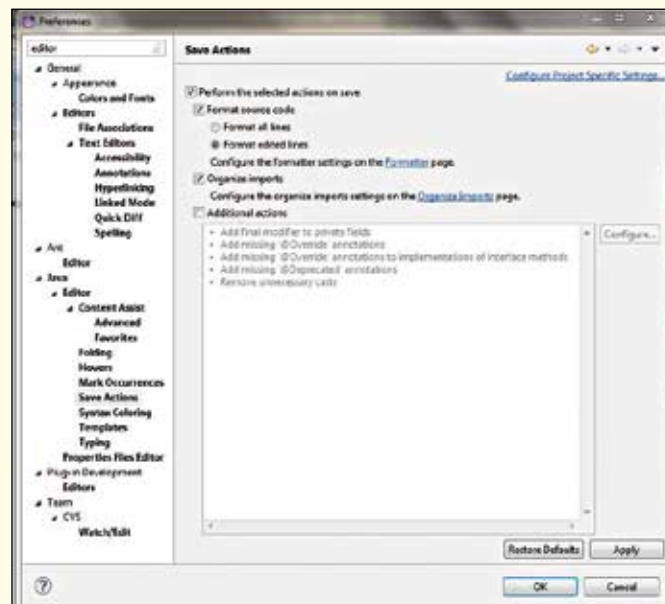


Enable Step filtering for debugging

This small change can make quite the difference for a programmer. Using Step filters, you can sieve out all the unnecessary steps you don't want to see during program debugging. Now while this looks like an insignificant tip, it might help you to stay focused on the actual useful bit of the code since you don't need to pay heed to internal actions of the software IDE which don't concern you. Enable the 'Use step filters' option by going to Preferences > Java > Debug > Step Filtering

BIT TORRENT

Unless you've been living in a cave, you must have heard of torrents. Many of you may have also used them, considering the ease by which we can



Format edited lines like never before